

BESONC — Full Core Platform Plan

Multi-Delivery Ecosystem for Ghana

Version 1.0 — Core Logic Only

SECTION 1: PLATFORM IDENTITY

What Besonc Is: A multi-service delivery platform built for Ghana that lets customers order food, groceries, shop items, market goods, pharmacy products, and laundry services from vendors — and request errands and parcel deliveries — all fulfilled by riders.

Three User Types:

- **Customers** — order and receive
- **Vendors** — list products/services, receive orders, prepare/fulfill
- **Riders** — pick up and deliver, run errands

Two Service Engines:

- **Catalogue-Based** — involves all 3 parties (customer, vendor, rider). Fixed prices. 6 services.
 - **Request-Based** — involves customer and rider only. No vendor. 2 services.
-

SECTION 2: ALL 8 SERVICES — DETAILED

ENGINE 1: CATALOGUE-BASED SERVICES (6 services, one template)

Core Principle: These 6 services share ONE order flow template. The only things that differ are the catalogue structure (what vendors list) and minor variations noted below.

SERVICE 1: FOOD

Purpose: Ready-to-eat meals & drinks

Vendor Types:

- Restaurants (local & international)
- Chop Bars
- Local food joints
- Fast food outlets
- Street food vendors
- Night vendors
- Home kitchens / Home cooks
- Bakeries & pastries
- Drinks & beverages (smoothies, juices, sobolo, ice cream)

Catalogue Structure:

```
Vendor (Restaurant)
└─ Category (e.g., "Rice Dishes", "Soups", "Drinks")
  └─ Item (e.g., "Jollof Rice")
    ├── Base Price: GHS 35
    ├── Description
    ├── Image
    ├── Available: true/false
    └─ Addon Groups
      ├── "Protein" (required, pick 1-3)
      │   ├── Chicken +GHS 15
      │   ├── Fish +GHS 12
      │   └─ Beef +GHS 10
      ├── "Extras" (optional, pick 0-3)
      │   ├── Extra plantain +GHS 5
      │   ├── Salad +GHS 8
      │   └─ Shito +GHS 0
      └─ "Drink" (optional, pick 0-1)
          ├── Sobolo +GHS 10
          └─ Water +GHS 5
```

Special Rules: None — this is the purest form of the template.

SERVICE 2: GROCERIES

Purpose: Daily household shopping

Vendor Types:

- Supermarkets
- Mini-marts & corner shops

Catalogue Structure:

```
Vendor (Supermarket)
└─ Category (e.g., "Fresh Produce", "Dairy", "Household")
  └─ Item (e.g., "Golden Tree Chocolate")
    ├── Base Price: GHS 18
    ├── Description
    ├── Image
    ├── Available: true/false
    ├── Unit: "per piece" / "per kg" / "per pack"
    ├── Quantity selector (1, 2, 3...)
    └─ Addon Groups: typically none
```

Difference from Food:

- Items have unit types (per piece, per kg, per pack, per bottle)
 - Quantity selector is more prominent (customer might buy 5 of something)
 - No addon groups typically — items are standalone
 - Vendor may offer **item substitution** preference: customer can set "allow substitution" or "refund if unavailable" per item
-

SERVICE 3: SHOP

Purpose: Non-food retail shopping

Vendor Types:

- Electronics & gadgets shops
- Phone & accessories shops
- Fashion (clothes, shoes, bags)
- Beauty & cosmetics
- Hair & salon supplies
- Stationery & books

Catalogue Structure:

```
Vendor (Shop)
└─ Category (e.g., "Phone Cases", "Earphones")
   └─ Item (e.g., "iPhone 15 Clear Case")
      ├── Base Price: GHS 85
      ├── Description
      ├── Images (multiple)
      ├── Available: true/false
      ├── Quantity selector
      └─ Variant Groups
         └─ "Color" (required, pick 1)
            ├── Black
            ├── Clear
            └─ Blue
```

Difference from Food:

- Items may have **variants** (size, color) instead of addons
- Multiple images per item
- **Payment Rule: ALL shop orders are prepaid only. No Cash on Delivery.** High-value items cannot risk COD non-acceptance.

SERVICE 4: MARKET

Purpose: Traditional African market shopping (Makola, Kejetia, Kaneshie, Kotokuraba, etc.)

This is the service that needed an identity decision. Here is the verdict:

Market is HYBRID – it is catalogue-based but with a structured shopping list option.

How it works:

Option A – Catalogue Browse (for onboarded market vendors): Market vendors who are onboarded list their products with prices.

```
Vendor (Auntie Ama - Makola Market)
└─ Category (e.g., "Fresh Produce", "Grains")
   └─ Item (e.g., "Fresh Tomatoes")
      ├── Price: GHS 30
      ├── Unit: "per bowl" / "per basket" / "per bag"
      ├── Image
      ├── Available: true/false
      └─ Quantity selector
```

Option B – Shopping List (for non-onboarded market sellers): Customer creates a structured shopping list and selects a target market.

```
Shopping List Order:
└─ Target Market: Makola Market
   └─ Items:
      ├── "Tomatoes - 1 big bowl"
      ├── "Onions - 1 bag"
      ├── "Fresh tilapia - 5 pieces"
      ├── "Garden eggs - 10 pieces"
      └─ "Kontomire - 2 bundles"
   └─ Special Instructions: "Pick the firm tomatoes, not soft ones"
```

When customer submits a shopping list:

- System estimates the cost based on known average market prices for the items + delivery fee
- Customer pays the estimated amount (pre-authorization model – same as errands)
- Rider is assigned and goes to the market to buy
- If actual cost differs, top-up or refund is handled

Why Hybrid: This allows Besonc to serve market orders immediately at launch (via shopping list) even before market vendors are onboarded, while also providing a richer catalogue experience as vendors join over time.

Shopping list market orders use the errand pricing model (pre-auth + settlement) because actual market prices fluctuate. Catalogue market orders use the standard fixed-price model.

SERVICE 5: PHARMACY & HEALTH

Purpose: Medicine & wellness items

Vendor Types:

- Licensed / verified pharmacies only

Catalogue Structure:

```
Vendor (Pharmacy)
└─ Category (e.g., "Pain Relief", "First Aid", "Baby Care")
    └─ Item (e.g., "Paracetamol 500mg")
        ├── Price: GHS 12
        ├── Description
        ├── Image
        ├── Available: true/false
        ├── Requires Prescription: true/false
        ├── Quantity selector
        └─ Substitution Allowed: true/false
```

Difference from Food:

- **Prescription Upload:** Customer can upload a prescription image at checkout. If ANY item in the cart is marked `requires_prescription: true`, upload becomes mandatory.
- **Extra State in Order Flow:** Order goes to `prescription_review` state before `vendor_accepted`. The pharmacist reviews the prescription and either accepts, rejects, or suggests alternatives.
- **Substitution Control:** Some medicines cannot be substituted without pharmacist approval. Customer can flag substitution preference per item.
- **Vendor Verification:** Only admin-verified pharmacies can list on this service. Stricter onboarding.

SERVICE 6: LAUNDRY PICKUP & DROP-OFF

Purpose: Clothing care logistics

Vendor Types:

- Laundry services
- Dry cleaning services

Catalogue Structure:

```
Vendor (Laundry)
└─ Service Type (e.g., "Wash & Fold", "Wash Iron & Fold", "Dry Cleaning")
    └─ Item (e.g., "Shirt")
        ├── Price: GHS 8
        └─ Quantity selector

OR

└─ Pricing Model: "Per Bag"
    ├── Small Bag: GHS 50
    ├── Medium Bag: GHS 80
    └─ Large Bag: GHS 120
```

Critical Difference from All Other Catalogue Services: Laundry involves **TWO trips** instead of one.

- **Trip 1 (Pickup):** Rider picks up dirty laundry from customer → delivers to laundry vendor
- **Trip 2 (Return):** After vendor processes, rider picks up clean laundry from vendor → delivers back to customer

This means:

- Two rider assignments (can be different riders)
- Two delivery fees
- A processing period in between (could be hours or days)
- Vendor sets estimated processing time (e.g., "24 hours", "48 hours")

Pricing:

- Service cost (set by vendor) + Pickup delivery fee + Return delivery fee
- Both delivery fees calculated at checkout so customer sees full cost upfront

ENGINE 2: REQUEST-BASED SERVICES (2 services, customer + rider only)

Core Principle: No vendor is involved. Customer makes a request, system estimates cost, rider is assigned to fulfill it. Communication between customer and rider is constant throughout.

SERVICE 7: PARCEL & COURIER DELIVERY

Purpose: Sending items & documents from point A to point B

Sub-categories:

- Small parcels
- Medium packages
- Fragile items
- Confidential items
- Documents (school, bank, legal, office, visa/embassy)

How it works:

Customer fills form:

```
└─ Pickup Location: [GPS pin + description]
└─ Pickup Contact: [Name + Phone]
└─ Drop-off Location: [GPS pin + description]
└─ Drop-off Contact: [Name + Phone]
└─ Package Category: [Small parcel / Medium / Fragile / Document / Confidential]
└─ Weight Estimate: [0-1kg / 1-5kg / 5-10kg / 10-20kg]
└─ Description: "Brown envelope with school documents"
└─ Fragile: yes/no
└─ Special Instructions: "Call before arriving"
```

Pricing is FIXED at checkout — calculated from:

- Weight category
- Distance between pickup and drop-off
- Package type surcharge (fragile, confidential)
- Platform fee

Formula:

Parcel Price = Base Fee (by weight category) + (Distance in km × Per-km Rate) + Surcharge (if fragile/confidential) + Platform Fee



Customer sees exact price before paying. No estimates.

SERVICE 8: ERRANDS & PERSONAL ASSISTANT

Purpose: "Do it for me" — rider performs a task on behalf of the customer

Sub-categories:

- Buy & deliver items (from any location)
- Bill payments (ECG, Water, Internet)
- Pick up forgotten items
- Queue services (where allowed)
- General personal errands
- School & hostel errands (provisions, stationery, food, parent-to-student deliveries, forgotten items)

How it works:

Customer fills form:

```
└─ Errand Type: [Buy & Deliver / Bill Payment / Pickup / Queue / General / School]
└─ Description: "Buy 3 packs of Indomie and 6 eggs from any shop near Legon"
└─ Task Location: [GPS pin – where rider should go] (optional for some types)
└─ Delivery Location: [GPS pin – where to deliver result]
└─ Budget Items (structured list, optional):
  |   └─ "Indomie × 3 packs – ~GHS 30"
  |   └─ "Eggs × 6 – ~GHS 18"
  |   └─ "Total items estimate: ~GHS 48"
└─ Special Instructions: "Any brand of egg is fine"
└─ Attached Images (optional): [photo of item needed, prescription, etc.]
```

Pricing is ESTIMATED at checkout — uses pre-authorization model:

Errand Cost = Estimated Item Cost + Errand Service Fee + Delivery Fee + Platform Fee

Customer pays: Full estimated amount

Settlement:

- If rider spends LESS → difference refunded to customer wallet
- If rider spends MORE (within 15% tolerance) → charged automatically
- If rider spends MORE (beyond 15%) → rider sends top-up request via app → customer approves → additional charge processed

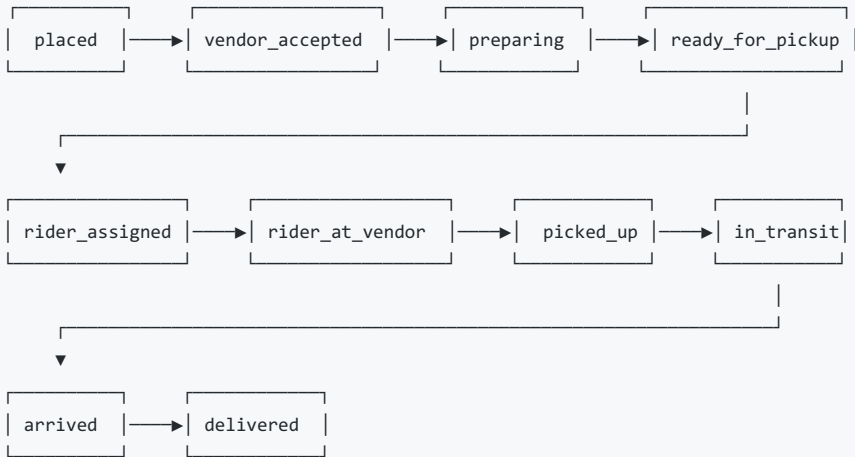
Rider provides proof:

- Rider uploads photos of receipts after purchase
- Rider uploads photos of items bought
- These are visible to customer in real-time

SECTION 3: ORDER STATE MACHINES

This is the single source of truth for every order's lifecycle. Every notification, every screen update, every payment action is triggered by a state transition.

State Machine A: Standard Catalogue (Food, Groceries, Shop, Market-Catalogue, Pharmacy without prescription)



State Definitions:

State	Who Triggers	What Happens
placed	System (after payment)	Order created, vendor notified (push + sound)
vendor_accepted	Vendor taps "Accept"	Customer notified, dispatch service starts looking for rider
preparing	Vendor taps "Preparing"	Customer sees "Your order is being prepared"
ready_for_pickup	Vendor taps "Ready"	Dispatch service assigns rider (if not already assigned during preparing)
rider_assigned	System (dispatch)	Customer + vendor notified with rider details
rider_at_vendor	Rider taps "Arrived at vendor" OR geo-fence auto-detect	Vendor notified to hand over order
picked_up	Rider taps "Picked up"	Customer notified, live tracking begins
in_transit	Automatic after picked_up	Customer sees rider moving on map
arrived	Rider taps "Arrived" OR geo-fence auto-detect	Customer notified "Rider is at your location"
delivered	Rider taps "Delivered" + confirms	Order complete. Payment settled. Customer can now rate.

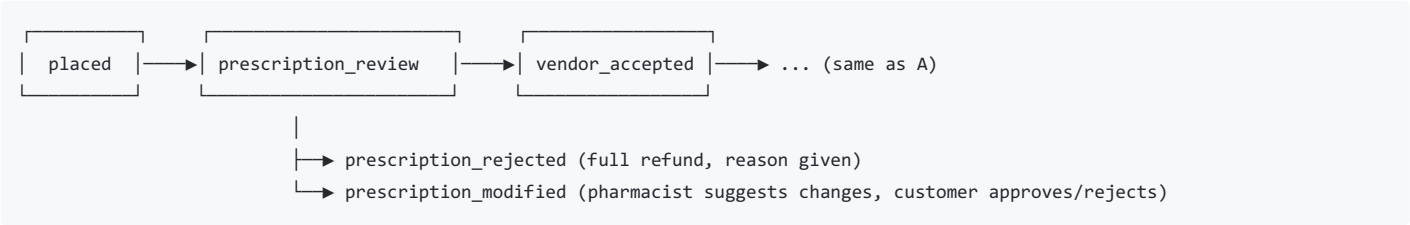
Failure States (can happen from multiple states):

State	Who Triggers	What Happens
vendor_rejected	Vendor taps "Reject" (from placed)	Customer notified + full refund
vendor_cancelled	Vendor cancels (from accepted or preparing)	Customer notified + full refund, reason required
customer_cancelled	Customer cancels (rules apply based on current state)	See Cancellation Rules section
rider_unassigned	System (all riders declined or timed out)	Retry dispatch or notify customer
failed	System or admin	Something went wrong, manual resolution needed

When does rider assignment happen? Rider assignment begins at vendor_accepted state. The system starts looking for riders immediately. Ideally, rider is assigned before ready_for_pickup . If no rider is assigned by ready_for_pickup , dispatch retries with broader radius.

State Machine B: Pharmacy with Prescription

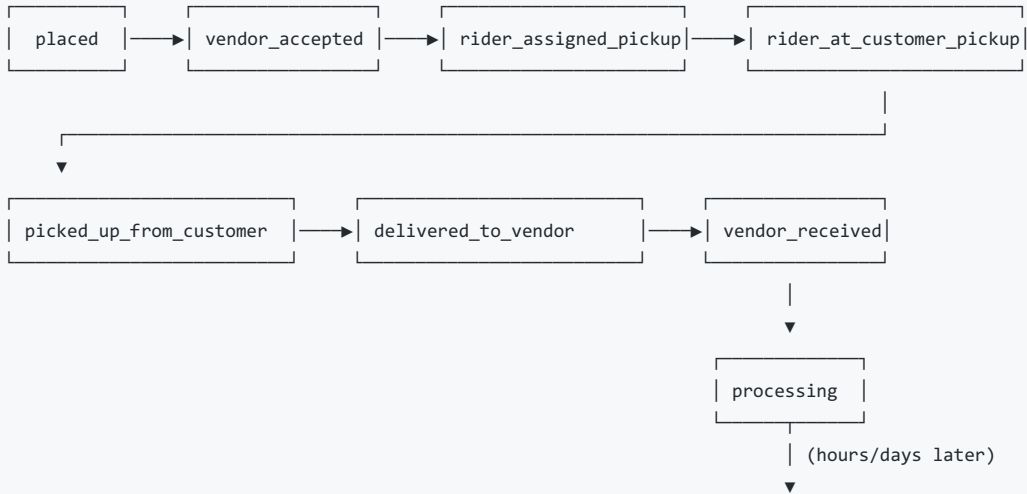
Same as State Machine A but with one extra state at the beginning:



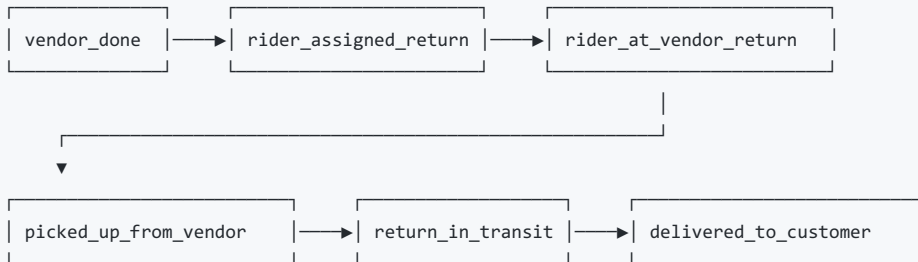
State	Who Triggers	What Happens
prescription_review	System (auto, if order has prescription items)	Pharmacist reviews uploaded prescription
prescription_rejected	Pharmacist	Customer notified with reason, full refund
prescription_modified	Pharmacist	Customer sees suggested changes, can accept (→ vendor_accepted) or reject (→ full refund)

State Machine C: Laundry (Two Trips)

TRIP 1 – PICKUP FROM CUSTOMER:



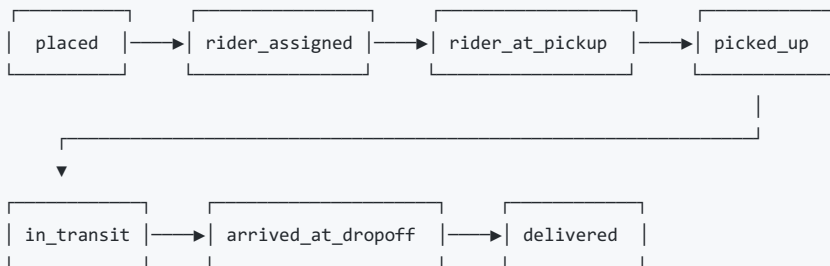
TRIP 2 – RETURN TO CUSTOMER:



Key Details:

- Trip 1 rider and Trip 2 rider can be **different people**
- `processing` state has an estimated completion time set by the vendor (e.g., 24 hours)
- When vendor taps "Done processing" → `vendor_done` → dispatch assigns rider for return trip
- Customer is notified at every state transition
- Payment is captured at `placed` but vendor settlement happens after `delivered_to_customer`

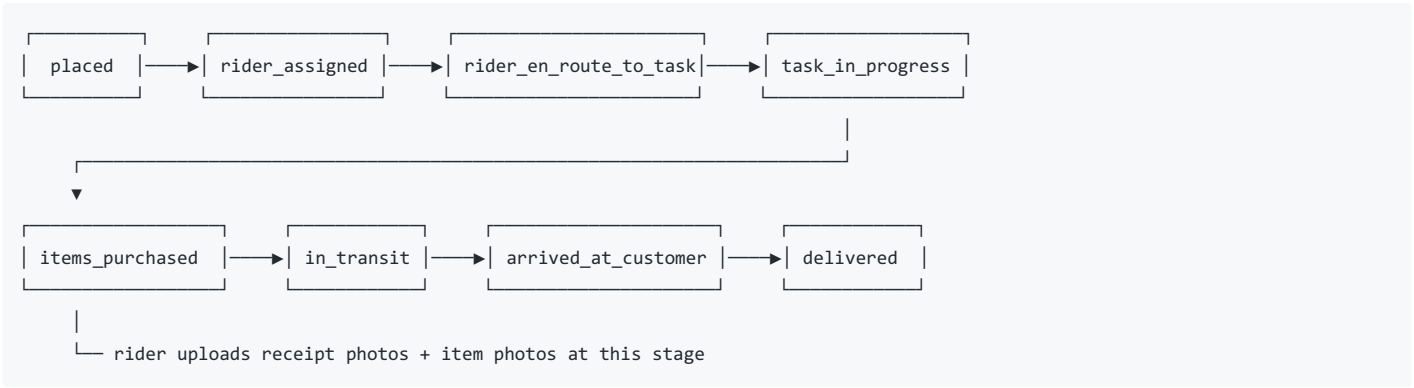
State Machine D: Parcel



Key Details:

- No vendor involved – rider goes directly to pickup location
- Pickup contact (sender) and drop-off contact (receiver) are both notified
- Rider assignment starts immediately at `placed` since there's no vendor to accept
- Receiver gets SMS notification with tracking link even if they don't have the app

State Machine E: Errand



Extra Errand States:

State	Who Triggers	What Happens
topup_requested	Rider (from task_in_progress)	Actual cost exceeds estimate by >15%. Rider sends topup request with amount + photo evidence. Customer approves or rejects.
topup_approved	Customer	Additional amount charged. Rider continues.
topup_rejected	Customer	Rider and customer discuss via chat. Rider may adjust purchase or customer cancels remaining items.
item_unavailable	Rider (from task_in_progress)	Specific item not available. Rider notifies customer with substitution suggestion. Customer approves substitution or removes item.

SECTION 4: RIDER ASSIGNMENT (DISPATCH) LOGIC

Assignment Model: Broadcast to Nearest, First-to-Accept-Wins

How It Works

- Step 1: Order needs a rider
- └ Catalogue: triggered at vendor_accepted
 - └ Parcel: triggered at placed
 - └ Errand: triggered at placed
- Step 2: System finds online riders within radius
- └ Initial radius: 3km from pickup location
 - └ Filters:
 - | └ Must be online (toggled on)
 - | └ Must not be on an active delivery (unless batching is enabled later)
 - | └ Must have appropriate vehicle for order type
 - | └ Must not have excessive unremitted COD balance
 - └ Sort by: nearest first
- Step 3: Broadcast to nearest 3 riders simultaneously
- └ Each rider gets push notification + sound alert
 - └ Rider sees: pickup location, drop-off area (not exact), estimated earnings, order type
 - └ Rider does NOT see: exact items, customer details (until accepted)
- Step 4: First rider to tap "Accept" wins
- └ Assignment confirmed
 - └ Other riders' offers cancelled
 - └ Customer + vendor notified with rider details (name, photo, vehicle, phone)
 - └ Timer: 30 seconds per round
- Step 5: If no rider accepts in 30 seconds → Round 2
- └ Expand radius to 5km
 - └ Broadcast to next 3 nearest riders
 - └ Timer: 30 seconds
- Step 6: If no rider accepts → Round 3
- └ Expand radius to 8km
 - └ Broadcast to next 3 nearest riders
 - └ Timer: 30 seconds
- Step 7: If no rider accepts after 3 rounds (90 seconds total)
- └ Notify customer: "Finding a rider is taking longer than usual"
 - └ System retries every 60 seconds for up to 5 minutes
 - └ After 5 minutes with no rider: customer gets option to wait or cancel (full refund)

Rider Vehicle Types & Order Matching

Vehicle Type	Can Handle
Bicycle	Small parcels, documents, small food orders
Motorbike	All food, groceries, parcels up to 10kg, errands, pharmacy
Car	Large groceries, bulk market orders, laundry, parcels up to 20kg, fragile items

System only broadcasts to riders with appropriate vehicle type for the order.

Rider Reassignment

If an assigned rider cancels after accepting:

- System immediately starts a new assignment cycle
- Customer and vendor are notified: "New rider being assigned"
- Rider who cancelled gets a cancellation count (too many = temporary suspension)

SECTION 5: ADDRESS & LOCATION SYSTEM (GHANA-SPECIFIC)

The Ghana Address Problem

Most of Ghana does not have reliable street addresses. Customers navigate by landmarks, descriptions, and GPS. Besonc must handle this reality.

Address Structure

```
Address Object:
├─ label: "Home" / "Work" / "School" / custom
├─ coordinates: { lat: 5.6037, lng: -0.1870 } ← PRIMARY (from map pin)
├─ ghanapost_address: "GA-123-4567" ← OPTIONAL
├─ area_name: "Osu" ← from reverse geocoding
├─ landmark: "Behind the MTN mast, near Papaye" ← user typed
├─ delivery_instructions: "Call when you arrive, blue gate on the left" ← user typed
└─ contact_phone: "+233XXXXXXXX" ← for this specific address
```

How Address Selection Works

Setting delivery address (customer):

1. App opens map centered on user's current GPS location
2. User can:
 - o a) Use current location (GPS auto-detect) → confirm pin position
 - o b) Search by area/place name (powered by Google Places API) → adjust pin
 - o c) Enter GhanaPostGPS digital address → system converts to coordinates → show on map → confirm
 - o d) Select from saved addresses
3. After pin is set, user fills:
 - o Landmark (strongly encouraged, shown as prominent field)
 - o Delivery instructions (optional but encouraged)
4. Address is saved for future use (user can choose to save or not)

For vendors: Vendor address is set once during onboarding. Fixed location. Includes coordinates + landmark.

For parcels: Both pickup and drop-off addresses follow the same structure above.

Distance Calculation

- All distance calculations use **road distance** (Google Directions API), not straight-line distance
- Used for: delivery fee calculation, rider assignment radius, ETA estimation
- Fallback: if Google Directions fails, use straight-line distance × 1.4 (average road winding factor for Ghanaian cities)

Map Provider

- **Google Maps SDK** for mobile (Flutter) and web (NextJS)
- **Google Places API** for search/autocomplete
- **Google Directions API** for routing and distance
- Budget consideration: cache frequently requested routes (e.g., vendor → common delivery areas) in Redis to reduce API calls

SECTION 6: PRICING ENGINE

Price Components for Every Order

```
What customer pays at checkout:
├─ Item Total (sum of all items × quantities + addons/variants)
├─ Delivery Fee
├─ Platform Service Fee
└─ Total = Item Total + Delivery Fee + Service Fee
```

Delivery Fee Calculation

Delivery Fee = Base Fee + (Distance × Per-km Rate)

Tiers:

- └─ 0 - 3km: Base GHS 5 + GHS 1.5/km
- └─ 3 - 7km: Base GHS 8 + GHS 1.2/km
- └─ 7 - 15km: Base GHS 12 + GHS 1.0/km
- └─ 15km+: Base GHS 18 + GHS 0.8/km

Surcharges:

- └─ Peak hours (11am-1pm, 6pm-9pm): +20%
- └─ Rain/bad weather: +15% (triggered manually by admin or automatically via weather API)
- └─ Night delivery (9pm - 6am): +25%
- └─ Fragile/heavy item: +GHS 5-10 (based on category)

Note: These are starting values. Admin can adjust all pricing parameters from the admin panel without code changes. Store pricing rules in a configuration service/database, not hardcoded.

Platform Service Fee

Service Fee = Percentage of Item Total

- └─ Food: 5% of item total (min GHS 2, max GHS 15)
- └─ Groceries: 4%
- └─ Shop: 5%
- └─ Market (catalogue): 5%
- └─ Market (shopping list): 7% (higher because platform manages the buying)
- └─ Pharmacy: 4%
- └─ Laundry: 5%
- └─ Parcel: Flat GHS 3 (no percentage, already priced by weight+distance)
- └─ Errand: 8% of estimated task cost

Platform Revenue (Commission from Vendors)

For catalogue services, platform takes commission from vendor earnings:

- └─ Food: 15% of item total
- └─ Groceries: 12%
- └─ Shop: 10%
- └─ Market: 15%
- └─ Pharmacy: 10%
- └─ Laundry: 12%

Vendor receives: Item Total - Platform Commission

Rider receives: Delivery Fee (full)

Platform receives: Platform Commission + Service Fee

Parcel Pricing (Fixed)

Base Fee by Weight:

- └─ 0 - 1kg (Document/Small): GHS 10
- └─ 1 - 5kg (Small Parcel): GHS 15
- └─ 5 - 10kg (Medium): GHS 25
- └─ 10 - 20kg (Large): GHS 40

+ Distance Fee (same tiers as delivery fee above)

+ Surcharge: Fragile +GHS 10, Confidential +GHS 5

Rider receives: 80% of total parcel fee

Platform receives: 20% of total parcel fee

Errand Pricing (Estimated → Settled)

Errand Cost = Estimated Item Cost + Errand Service Fee + Delivery Fee

- └ Estimated Item Cost: Based on customer's budget list
- └ Errand Service Fee: GHS 15 flat (for the rider's effort/time)
- └ Delivery Fee: Distance-based (task location → customer location)

Customer pays: Full estimated total
After completion: Settlement based on actual receipts

Rider receives: Errand Service Fee + Delivery Fee
Platform receives: 8% of estimated task cost
Customer: Refunded difference or charged overage (within 15% auto, beyond = approval needed)

SECTION 7: PAYMENT SYSTEM

Payment Methods

Method	Pay for Orders	Receive Payouts	Top-up Wallet
Mobile Money (MTN MoMo, Vodafone Cash, AirtelTigo Money)	☑	☑	☑
Bank Card (Visa, Mastercard)	☑	☑	☑
Bank Transfer	☑	☑	☑
Cash on Delivery	☑ (eligible orders only)	N/A	N/A
Besonc Wallet	☑	N/A	N/A

Payment Gateway: Paystack (handles all mobile money, card, and bank operations in Ghana)

Wallet System

Every user has a wallet. The wallet holds real money (GHS).

Customer Wallet:

- Funded by: refunds, cancelled order reversals
- Can be used to: pay for orders (full or partial), combined with other payment methods
- Can be withdrawn to: mobile money
- Cannot be topped up directly at launch (only receives refunds) — direct top-up is a Phase 2 feature

Vendor Wallet:

- Funded by: order settlements (after successful delivery)
- Can be used to: pay for premium subscription (later phase)
- Can be withdrawn to: mobile money or bank
- Settlement timing: credited to wallet immediately after order is marked `delivered`, but **withdrawal hold period of 24 hours** (protects against disputes)

Rider Wallet:

- Funded by: delivery fee earnings per completed order
- Debited by: COD remittance obligations
- Can be withdrawn to: mobile money
- Settlement timing: credited immediately after order is marked `delivered`
- **COD balance:** tracked separately. If rider collects cash, they owe the platform. This is deducted from earnings or rider remits via mobile money.

Minimum withdrawal: GHS 20 **Withdrawal fee:** GHS 1 (flat, regardless of amount)

Double-Entry Ledger

Every money movement creates TWO entries — a debit and a credit. No money appears or disappears.

Ledger Accounts

Platform accounts:

- └─ PLATFORM_REVENUE (credits: commissions + service fees)
- └─ PLATFORM_HOLDING (temporarily holds customer payments before settlement)
- └─ PLATFORM_CASH_HOLDING (holds COD obligations)

Per-user accounts:

- └─ CUSTOMER_WALLET_{id}
- └─ VENDOR_WALLET_{id}
- └─ RIDER_WALLET_{id}

External accounts (represent money entering/leaving the system):

- └─ PAYSTACK_INFLOW (customer payments coming in)
- └─ PAYSTACK_OUTFLOW (withdrawals going out)

Example Ledger Entries — Prepaid Food Order

Customer orders GHS 50 of food. Delivery fee GHS 8. Service fee GHS 3. Total: GHS 61. Platform commission on food: 15% of GHS 50 = GHS 7.50

Entry 1 — Customer pays GHS 61 via mobile money:

Entry	Account	Debit	Credit
1a	PAYSTACK_INFLOW		GHS 61
1b	PLATFORM_HOLDING	GHS 61	

Entry 2 — Order delivered, settlement:

Entry	Account	Debit	Credit
2a	PLATFORM_HOLDING		GHS 61
2b	VENDOR_WALLET	GHS 42.50	
2c	RIDER_WALLET	GHS 8	
2d	PLATFORM_REVENUE	GHS 10.50	

Every cedi is accounted for. Debits always equal credits.

Example Ledger Entries — COD Order

Same order but customer pays cash to rider:

Entry 1 — Order placed (no money enters system yet):

Entry	Account	Debit	Credit
1a	RIDER_COD_OBLIGATION	GHS 61	
1b	PLATFORM_CASH_HOLDING		GHS 61

Entry 2 — Rider remits GHS 61 via mobile money:

Entry	Account	Debit	Credit
2a	PAYSTACK_INFLOW		GHS 61
2b	RIDER_COD_OBLIGATION		GHS 61
2c	PLATFORM_CASH_HOLDING	GHS 61	
2d	PLATFORM_HOLDING		GHS 61

Entry 3 — Settlement (same as prepaid Entry 2 above)

Cash on Delivery (COD) — Full Flow

Eligibility Rules

Not every order qualifies for COD. Rules:

Rule	Condition
Order value limit	COD only for orders ≤ GHS 200
Service restriction	NO COD for Shop service (all shop orders are prepaid only)
Customer history	New customers (< 3 completed orders): COD limit GHS 50
Rider COD balance	Rider cannot be assigned COD orders if they have > GHS 300 unremitted COD balance
Time restriction	No COD for night orders (after 9pm) — higher risk

COD Rider Remittance Flow

1. Rider delivers order → collects cash from customer
 2. Rider's COD balance increases by order amount
 3. Rider sees COD balance on their home screen (always visible, like a debt counter)
 4. Rider taps "Remit" → chooses mobile money → pays the amount → Paystack processes
 5. COD balance decreases
 6. If COD balance > GHS 300 → rider cannot receive new COD assignments
 7. If COD balance is not cleared within 24 hours → rider gets warning notification
 8. If COD balance is not cleared within 48 hours → rider account temporarily suspended from new orders
 9. Admin can see all riders' COD balances and manually intervene

What if Customer Refuses to Pay (COD)?

1. Rider arrives, customer refuses to pay or is unreachable
 2. Rider taps "Customer unavailable / refused payment"
 3. Rider must wait 5 minutes (enforced by app timer)
 4. If still no resolution → rider taps "Return order"
 5. Order status → "return_to_vendor"
 6. Rider returns items to vendor
 7. Platform absorbs delivery cost or bills customer (based on admin policy)
 8. Vendor receives their items back, no settlement
 9. Customer gets a strike (3 strikes = COD privilege revoked)

SECTION 8: CANCELLATION & REFUND RULES

Customer Cancellation Rules

Current Order State	Can Cancel?	Refund	Penalty
placed (before vendor accepts)	✅ Yes	100% refund to original payment method	None
vendor_accepted	✅ Yes	100% refund	None (vendor hasn't started)
preparing	⚠️ Yes, but with cost	50% refund (vendor already working)	Customer pays 50% to vendor
ready_for_pickup	❌ No	No refund	—

picked up or later Current Order State	No Can Cancel?	No refund Refund	Penalty
--	--------------------------------------	--	--------------------------------

Refund destination: Refunds go to **Besonc Wallet** (instant) by default. Customer can request refund to original payment method (takes 24-48 hours via Paystack).

Vendor Cancellation Rules

Situation	What Happens
Vendor rejects order at <code>placed</code>	Full refund to customer. No penalty for first 5 rejections/month. After 5: warning. After 10: temporary suspension.
Vendor cancels at <code>preparing</code> (out of stock, etc.)	Full refund to customer. Vendor must provide reason. Platform tracks cancellation rate.
Vendor cancels repeatedly	Cancellation rate > 20% in a month → admin notified → possible suspension

Rider Cancellation Rules

Situation	What Happens
Rider declines broadcast offer	No penalty. Offer goes to next rider.
Rider cancels after accepting (before pickup)	Reassignment triggered. Rider gets cancellation count. 3 cancellations in a day = offline for 2 hours.
Rider cancels after pickup	Not allowed through app. Must contact support. Rare/emergency only.

SECTION 9: REAL-TIME INFRASTRUCTURE

Live Tracking

Component	Technology	Details
Rider location updates	WebSocket (Socket.IO)	Rider app sends GPS coordinates every 5 seconds when on active delivery, every 30 seconds when online but idle
Customer sees rider on map	WebSocket	Customer subscribes to their active order's tracking channel, receives rider location updates in real-time
ETA calculation	Google Directions API + caching	Recalculated every 30 seconds during active delivery. Cached aggressively.

In-App Chat

Component	Details
Customer ↔ Rider chat	Available from <code>rider_assigned</code> to <code>delivered</code> . Text messages + image sharing (for errands: receipts, item photos).
Customer ↔ Vendor chat	Available from <code>placed</code> to <code>delivered</code> . For clarifications about order (e.g., "no pepper please").
Implementation	Custom WebSocket-based chat service. Messages stored in Chat Service database.
Chat availability	Chat closes 30 minutes after order is <code>delivered</code> . After that, issues go through customer support.

In-App Calling (Masked)

Component	Details
Purpose	Rider needs to call customer for delivery directions. Customer needs to call rider.
Method	Masked phone numbers via Paystack's or a third-party masking service (e.g., Arkesel for Ghana). Neither party sees the other's real number.
Fallback	If masking service is unavailable, direct call with consent (both parties agreed to share number at order time).
Availability	Same window as chat: <code>rider_assigned</code> to 30 minutes after <code>delivered</code> .

Push Notifications

Event	Recipient	Channel
New order	Vendor	Push + in-app sound alert (loud, persistent until acknowledged)
Order accepted	Customer	Push
Rider assigned	Customer + Vendor	Push
Rider arriving	Customer	Push
Order delivered	Customer	Push
New assignment available	Rider	Push + sound
COD remittance reminder	Rider	Push + SMS
Order status updates	Customer	Push (every state transition)

Push provider: Firebase Cloud Messaging (FCM) for Android, Apple Push Notifications (APNs) for iOS – Flutter handles both via `firebase_messaging` package.

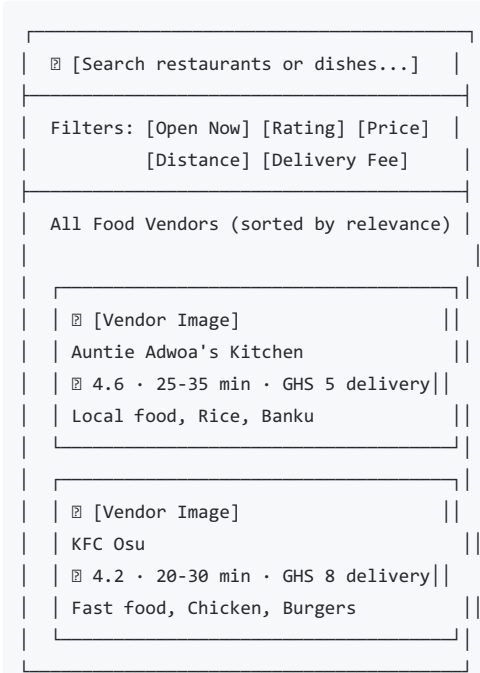
SMS fallback: For critical notifications (OTP, COD reminders, parcel recipient notifications), send SMS via Arkesel or Hubtel SMS API (Ghana-local providers with good delivery rates).

SECTION 10: SEARCH & DISCOVERY

Customer Home Screen Structure



When Customer Taps a Service (e.g., Food)



Search Logic

Search Type	What It Searches	Example
Global search (home screen)	Vendor names + item names across all services	"paracetamol" → shows pharmacy vendors with paracetamol
Service search (within a service)	Vendor names + item names within that service	Within Food: "jollof" → shows all restaurants with jollof

Item search (within a vendor) Search Type	Items in that vendor's catalogue What It Searches	Within KFC: "chicken" Example
--	--	----------------------------------

Search implementation:

- **Phase 1 (Launch):** PostgreSQL full-text search with `tsvector` — good enough for thousands of vendors and products
- **Phase 2 (Scale):** Elasticsearch when search volume demands it

Vendor Sorting Algorithm (Default)

Score = (0.3 × proximity_score) + (0.3 × rating_score) + (0.2 × order_volume_score) + (0.2 × preparation_speed_score)

Where:

proximity_score = inverse of distance (closer = higher)

rating_score = average rating normalized to 0-1

order_volume_score = total orders in last 30 days (normalized)

preparation_speed_score = average prep time (faster = higher)

Vendors who are currently closed are shown at the bottom, greyed out, with "Opens at [time]".

SECTION 11: VENDOR ONBOARDING & MANAGEMENT

Vendor Registration Flow

Step 1: Download vendor app → Sign up with phone number → OTP verification

Step 2: Select service type: [Food / Groceries / Shop / Market / Pharmacy / Laundry]

Step 3: Fill business details:

- └ Business name
- └ Business description
- └ Business location (GPS pin + landmark)
- └ Business phone number
- └ Owner name
- └ Owner Ghana Card number (for verification)
- └ Business registration number (optional at launch, required for pharmacy)
- └ Business images (storefront, interior – at least 2)
 - └ For Pharmacy: license number + license photo (mandatory)

Step 4: Submit for review → Status: "pending_review"

Step 5: Admin reviews and approves/rejects

- └ Approved → vendor can set up catalogue and go live
- └ Rejected → vendor notified with reason, can resubmit

Vendor Dashboard (Home Screen)

Auntie Adwoa's Kitchen

Open

(tap to close)

Today's Summary

Orders

12

Revenue

GHS 580

Rating

4.6

New Orders (3)

[See All]

← needs immediate attention

Order #1234 · 2 items · GHS 65

Accept

Reject

In Progress (2)

[See All]

Order #1230 · Preparing · 5min ago

Mark Ready for Pickup

Completed Today (7)

[See All]

Vendor Order Management Rules

Action	Time Limit	Consequence of Inaction
Accept/Reject new order	3 minutes	Auto-reject after 3 min, customer refunded, vendor gets inaction count
Update to "Preparing"	After accepting	Optional but encouraged for customer visibility
Update to "Ready for Pickup"	When done	Required — rider won't come until this is tapped
5+ auto-rejects in a day	—	Admin notified, possible temporary suspension

Vendor Menu/Catalogue Management

Vendor can:

- Add/edit/delete categories
- Add/edit/delete items with images, prices, descriptions
- Add addon groups and addon items (for food)
- Add variant groups (for shop)
- Toggle item availability (in stock / out of stock)
- Set preparation time estimate per item (optional)
- Bulk import catalogue via CSV (Phase 2)

Vendor Operating Hours

Vendor sets:

Regular hours: Mon-Fri 8am-9pm, Sat 9am-10pm, Sun closed

Can override for specific dates: "Dec 25 = closed"

Toggle: "I'm open now" / "I'm closed" (manual override regardless of schedule)

System behavior:

Outside hours → vendor shown as "Closed" to customers

Vendor manually closes → same

New orders not routed to closed vendors

Vendor Premium Subscription (Phase 2 — Structure Defined Now)

Free tier:

- └─ Standard listing
- └─ Up to 50 menu items
- └─ Basic analytics

Premium tier (monthly subscription):

- └─ Priority listing (shown higher in search results)
- └─ Unlimited menu items
- └─ Stories feature (WhatsApp-status-like, Phase 3)
- └─ Advanced analytics
- └─ Promotional tools
- └─ Reduced commission rate (e.g., 12% instead of 15%)

SECTION 12: RIDER ONBOARDING & MANAGEMENT

Rider Registration Flow

Step 1: Download rider app → Sign up with phone number → OTP verification

Step 2: Fill personal details:

- └─ Full name
- └─ Ghana Card number + photo (front & back)
- └─ Selfie (for identity matching)
- └─ Date of birth
- └─ Emergency contact
- └─ City/zone of operation
- └─ Phone number (already verified)

Step 3: Vehicle details:

- └─ Vehicle type: [Bicycle / Motorbike / Car]
- └─ For motorbike/car: license plate number
- └─ For motorbike/car: driver's license photo
- └─ Vehicle photo
- └─ Insurance details (if applicable)

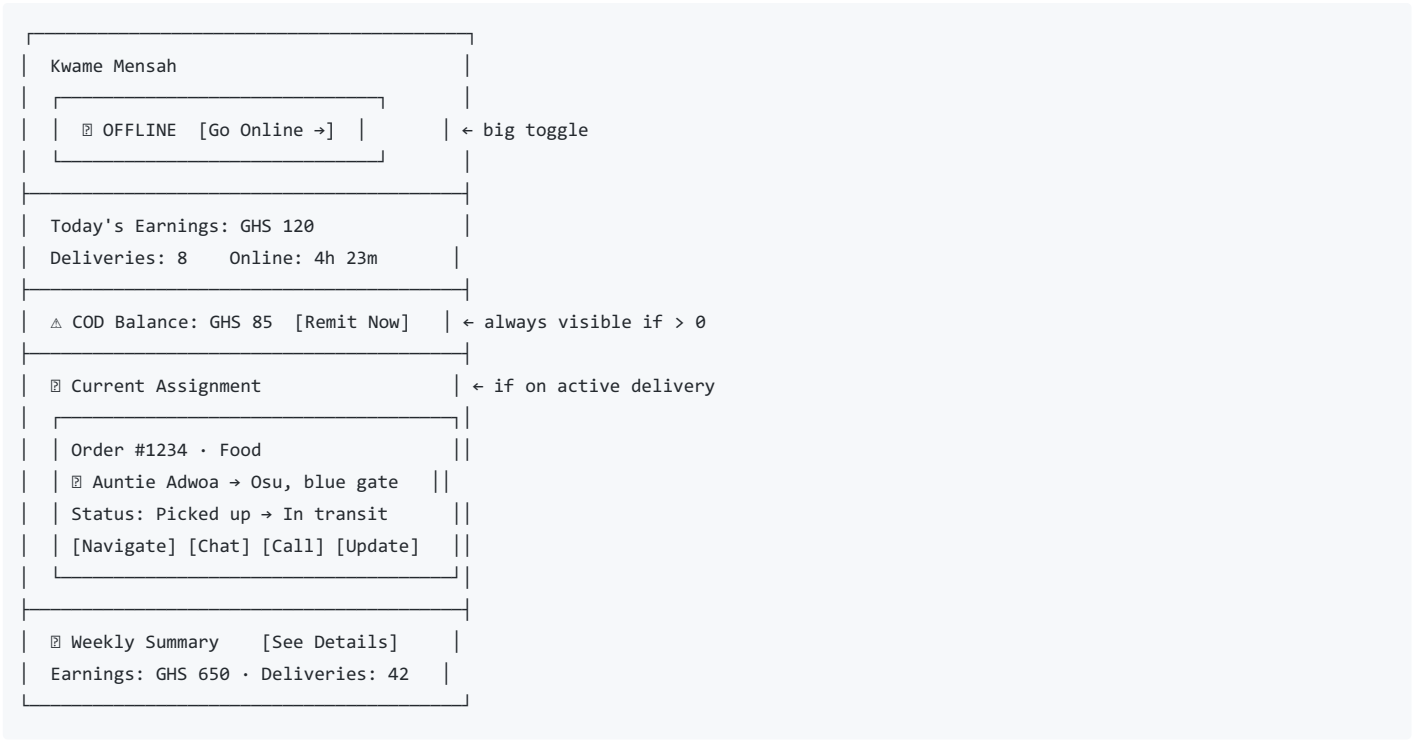
Step 4: Submit for review → Status: "pending_review"

Step 5: Admin reviews:

- └─ Verify Ghana Card (name matches, valid)
- └─ Verify license (for motorbike/car)
- └─ Verify vehicle photo
- └─ Approve / Reject with reason

Step 6: Approved → Rider can go online

Rider App — Home Screen



Rider Online/Offline

State	Behavior
Offline	Rider receives no order broadcasts. Not visible to dispatch.
Online	Rider is available. Receives order broadcasts. Location tracked every 30 seconds.
On delivery	Rider is on active order. Location tracked every 5 seconds. Does not receive new broadcasts (no batching at launch).
Auto-offline	If rider doesn't respond to any broadcast for 30 minutes, system auto-sets them offline and sends notification.

Rider Order Flow

1. Rider is online

2. Push notification: "New delivery available!"

3. Rider sees: pickup location, delivery area, estimated earnings, order type, distance

4. Rider taps [Accept] or [Decline] or timer expires (30 seconds)

5. If accepted:

a. Rider navigates to vendor (or pickup location for parcels)

b. At vendor: taps "Arrived at vendor"

c. Receives order from vendor: taps "Picked up"

d. Navigates to customer

e. At customer: taps "Arrived"

f. Hands over order: taps "Delivered"

g. If COD: collects cash → confirms amount → delivery complete

h. If prepaid: just confirms delivery → complete

6. Earning credited to rider wallet

7. Ready for next assignment

SECTION 13: MULTI-VENDOR CART BEHAVIOR

Rule: One Vendor Per Cart

At launch, a customer’s cart is restricted to **one vendor at a time**. This is the industry standard (Uber Eats, Bolt Food, Glovo all do this).

Why: Multiple vendors = multiple rider pickups = complex coordination, longer wait times, higher failure rate. Not worth the complexity at launch.

User Flow:

1. Customer adds items from Vendor A

2. Customer navigates to Vendor B and tries to add item

3. App shows: "You have items from [Vendor A] in your cart. Start a new cart?"
[Keep current cart] [Start new cart]

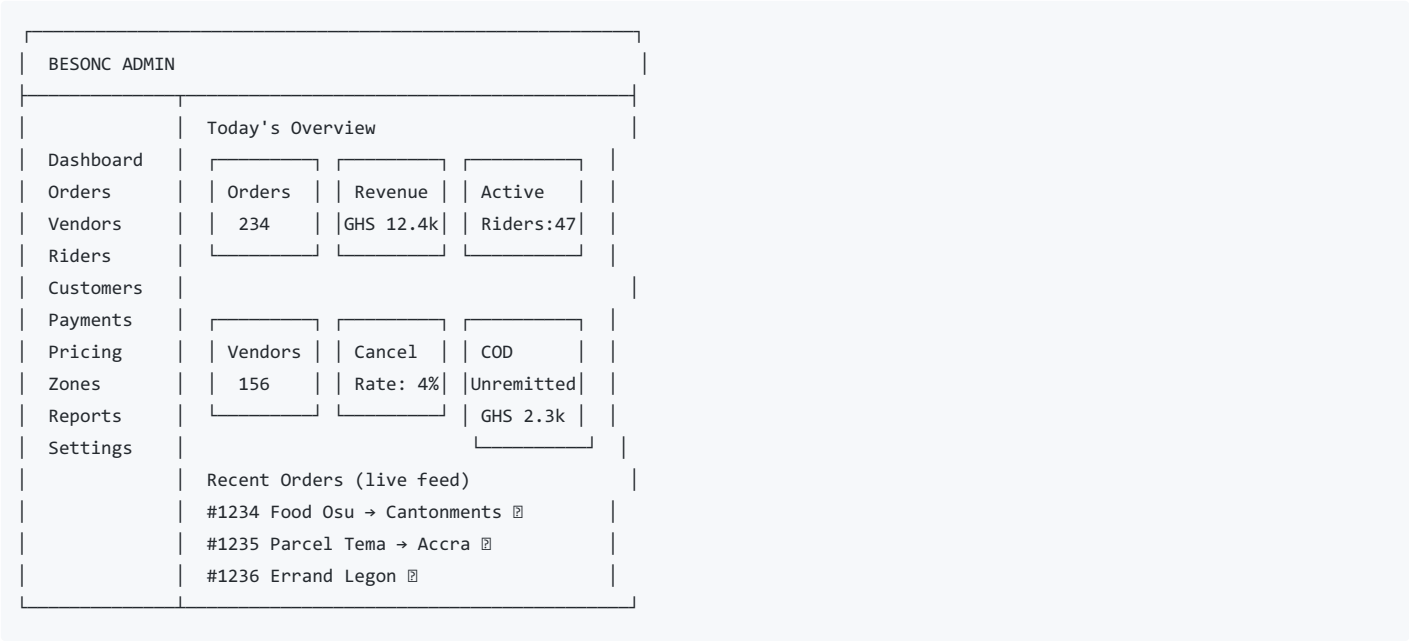
4. If "Start new cart" → clear cart, add Vendor B item

Phase 2 consideration: Allow multi-vendor orders that create separate sub-orders with separate riders, displayed to customer as one "group order" with combined tracking.

SECTION 14: ADMIN PANEL

Admin Web App (NextJS) – Core Features

Dashboard

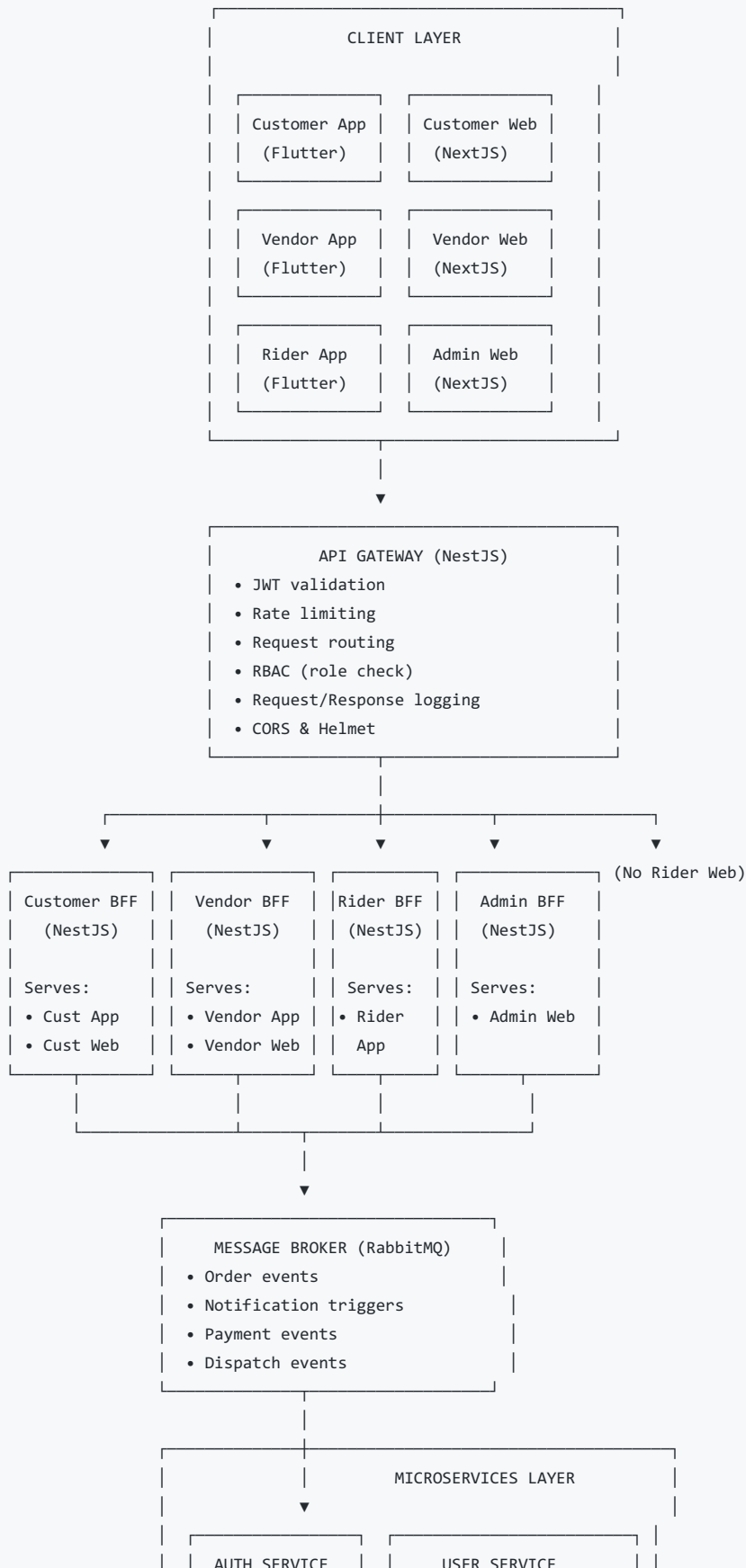


Core Admin Functions

Module	What Admin Can Do
Orders	View all orders, filter by status/service/date/vendor/rider. View order details. Manually change order status (emergency). Cancel orders with refund.
Vendors	View all vendors. Approve/reject pending applications. Suspend/reactivate vendors. View vendor performance metrics. Edit vendor details.
Riders	View all riders. Approve/reject applications. Suspend/reactivate. View COD balances. View performance. Force-offline a rider.
Customers	View all customers. View order history. Suspend accounts. View COD strike history.
Payments	View all transactions. View ledger entries. Reconciliation reports. Process manual refunds. View withdrawal requests. View unremitted COD.
Pricing	Edit delivery fee tiers. Edit commission rates. Edit service fees. Set surge pricing. Set COD limits. All without code changes.
Zones	Define delivery zones/areas. Set which areas are serviceable. Set zone-specific pricing if needed.
Reports	Daily/weekly/monthly revenue. Orders by service type. Vendor performance rankings. Rider performance rankings. Cancellation rates.

SECTION 15: TECHNOLOGY ARCHITECTURE

Full System Architecture



• Phone OTP • Google login • Apple login • JWT issue • 2FA • Passkeys	• Customer profiles • Vendor profiles • Rider profiles • Address management • Onboarding status • KYC/verification
DB: auth_db	DB: user_db

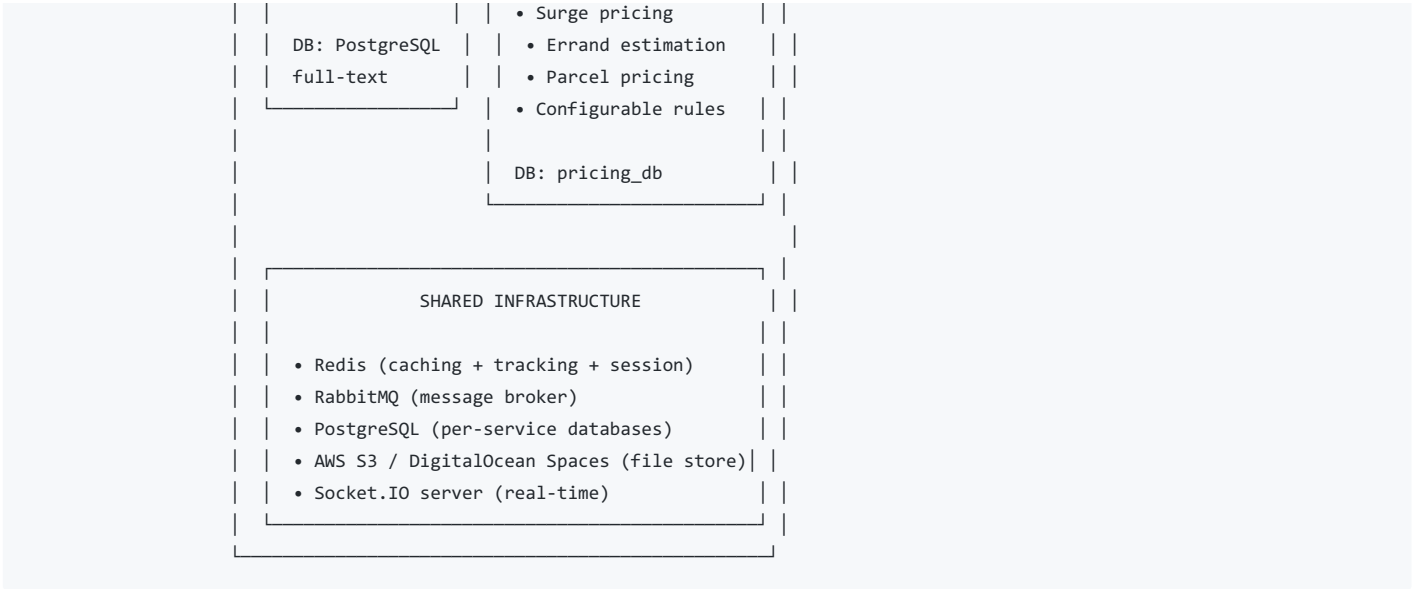
CATALOGUE SERVICE • Vendor stores • Categories & items • Addons & variants • Stock management • Operating hours • Service type configs	ORDER SERVICE • Cart • Order create • State machine • Order history • Cancellation • Substitution
ONE template, 6 services DB: catalogue_db	DB: order_db

DISPATCH SERVICE • Rider matching • Broadcast • Assignment • Reassignment • Radius logic	TRACKING SERVICE • Rider GPS ingestion • Location storage • Customer live feed • ETA calculation • Geo-fence detection
DB: dispatch_db	DB: tracking_db (Redis)

PAYMENT SERVICE • Paystack integration • Double-entry ledger • Wallet management • COD tracking • Withdrawals • Refund processing • Settlement logic • Idempotency	NOTIFICATION SVC • Push (FCM) • SMS (Arkesel) • In-app • Email (later) • Templates
DB: payment_db	DB:notification_db

CHAT SERVICE • WebSocket • Message store • Read receipts • Image sharing	MEDIA SERVICE • Image upload • Image compression • Receipt photos • Prescription uploads • Vendor/item images • Cloud storage (S3)
DB: chat_db	DB: media_db

SEARCH SERVICE • Vendor search • Item search • Filters • Sorting • Full-text	PRICING SERVICE • Delivery fee calc • Service fee calc • Commission calc
---	---



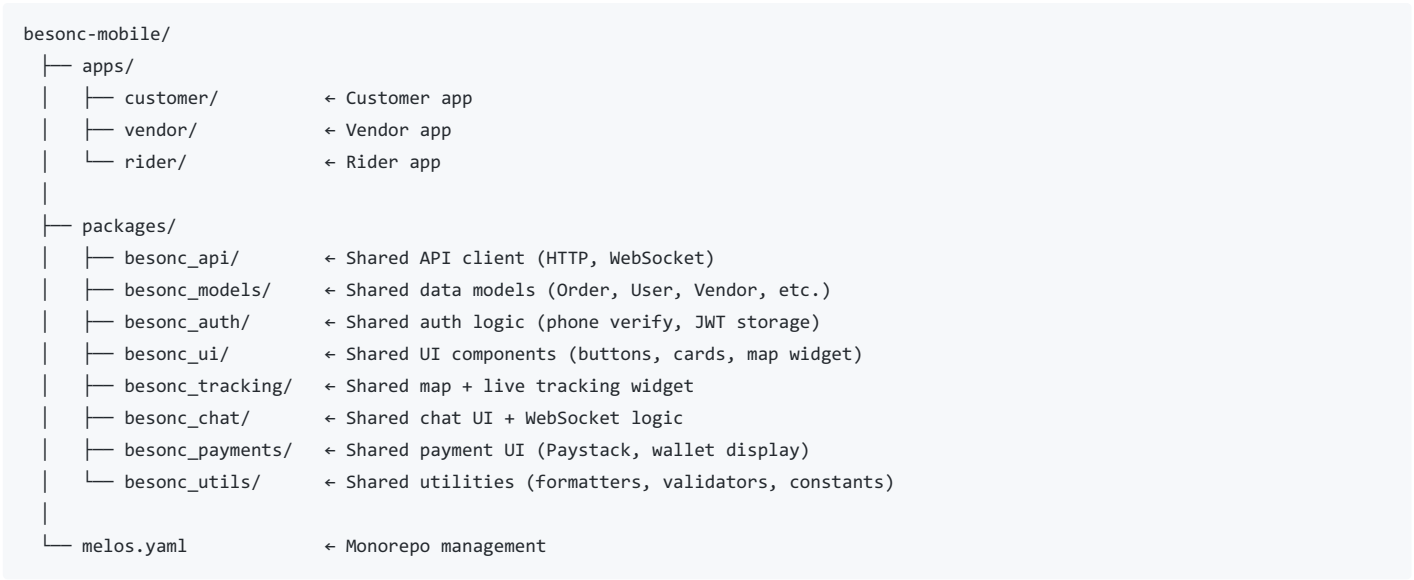
Microservice Count: 13 Services

#	Service	Database	Purpose
1	Auth Service	auth_db (PostgreSQL)	Authentication, OTP, tokens, 2FA, passkeys
2	User Service	user_db (PostgreSQL)	All user profiles, addresses, onboarding, KYC
3	Catalogue Service	catalogue_db (PostgreSQL)	Vendor stores, menus, products, stock, hours
4	Order Service	order_db (PostgreSQL)	Cart, orders, state machine, cancellations
5	Dispatch Service	dispatch_db (PostgreSQL)	Rider matching, broadcast, assignment
6	Tracking Service	tracking_db (Redis)	Rider GPS, live tracking, ETA, geo-fence
7	Payment Service	payment_db (PostgreSQL)	Ledger, wallets, Paystack, COD, settlements
8	Notification Service	notification_db (PostgreSQL)	Push, SMS, in-app notifications
9	Chat Service	chat_db (PostgreSQL)	Real-time messaging, image sharing
10	Media Service	media_db (PostgreSQL) + S3	File uploads, compression, storage
11	Search Service	Uses catalogue_db (read replica)	Vendor/item search, filters, sorting
12	Pricing Service	pricing_db (PostgreSQL)	All pricing calculations and rules
13	Admin Service	Uses other services' databases via APIs	Back-office operations, reports

Total: 13 microservices + 4 BFFs + 1 API Gateway = 18 deployable services

SECTION 16: SHARED FLUTTER ARCHITECTURE

Since you have 3 Flutter apps that share significant logic, structure them as a **monorepo with shared packages**:



What's shared vs. unique:

Shared (packages/)	Customer-only	Vendor-only	Rider-only
Auth flow	Home/discovery screen	Dashboard screen	Online/offline toggle
API client	Cart & checkout	Menu management	Assignment broadcast
Data models	Order tracking view	Order management	Navigation mode
Map/tracking widget	Address management	Operating hours	COD remittance
Chat UI	Service browsing	Stock management	Proof-of-delivery
Payment UI	Errand/parcel forms	Earnings view	Earnings view
Common widgets	Search	Analytics	—

SECTION 17: DATABASE SCHEMAS (Key Tables Only)

These are the core tables for the most critical services. Not exhaustive — but enough for your backend team to start.

User Service

```

-- All user types in one table with role differentiation
users
├─ id (UUID, PK)
├─ phone (VARCHAR, UNIQUE, NOT NULL)
├─ email (VARCHAR, NULLABLE)
├─ role (ENUM: customer, vendor_owner, rider, admin)
├─ first_name
├─ last_name
├─ profile_image_url
├─ phone_verified (BOOLEAN)
├─ status (ENUM: active, suspended, pending_review, rejected)
├─ created_at
└─ updated_at

addresses
├─ id (UUID, PK)
├─ user_id (FK → users)
├─ label (VARCHAR: "Home", "Work", custom)
├─ latitude (DECIMAL)
├─ longitude (DECIMAL)
├─ ghanapost_address (VARCHAR, NULLABLE)
├─ area_name (VARCHAR)
├─ landmark (TEXT)
├─ delivery_instructions (TEXT)
├─ contact_phone (VARCHAR)
├─ is_default (BOOLEAN)
└─ created_at

rider_profiles
├─ id (UUID, PK)
├─ user_id (FK → users)
├─ ghana_card_number (VARCHAR)
├─ ghana_card_image_url
├─ vehicle_type (ENUM: bicycle, motorbike, car)
├─ license_plate (VARCHAR, NULLABLE)
├─ drivers_license_url (NULLABLE)
├─ vehicle_image_url
├─ is_online (BOOLEAN)
├─ current_latitude (DECIMAL, NULLABLE)
├─ current_longitude (DECIMAL, NULLABLE)
├─ approved_at (TIMESTAMP)
└─ approved_by (FK → users, admin who approved)

```

Catalogue Service

```

stores
├─ id (UUID, PK)
├─ owner_id (FK → users via API)
├─ service_type (ENUM: food, groceries, shop, market, pharmacy, laundry)
├─ name (VARCHAR)
├─ description (TEXT)
├─ image_urls (JSON ARRAY)
├─ latitude (DECIMAL)
├─ longitude (DECIMAL)
├─ landmark (TEXT)
├─ phone (VARCHAR)
├─ is_active (BOOLEAN)
├─ is_open (BOOLEAN)
├─ operating_hours (JSON) -- {"mon": {"open": "08:00", "close": "21:00"}, ...}
├─ average_rating (DECIMAL, default 0)
├─ total_orders (INTEGER, default 0)
├─ average_prep_time_minutes (INTEGER)
├─ pharmacy_license_number (VARCHAR, NULLABLE)
└─ status (ENUM: pending_review, approved, suspended, rejected)

```

```

└─ status (ENUM: pending_review, approved, suspended, rejected)
└─ created_at

categories
└─ id (UUID, PK)
└─ store_id (FK → stores)
└─ name (VARCHAR)
└─ sort_order (INTEGER)
└─ is_active (BOOLEAN)

items
└─ id (UUID, PK)
└─ store_id (FK → stores)
└─ category_id (FK → categories)
└─ name (VARCHAR)
└─ description (TEXT)
└─ image_url (VARCHAR)
└─ base_price (DECIMAL)
└─ unit (VARCHAR: "per piece", "per kg", "per pack" – for groceries/market)
└─ is_available (BOOLEAN)
└─ requires_prescription (BOOLEAN, default false – for pharmacy)
└─ substitution_allowed (BOOLEAN, default true)
└─ sort_order (INTEGER)
└─ created_at

addon_groups
└─ id (UUID, PK)
└─ item_id (FK → items)
└─ name (VARCHAR: "Protein", "Extras")
└─ is_required (BOOLEAN)
└─ min_selections (INTEGER)
└─ max_selections (INTEGER)
└─ sort_order

addon_items
└─ id (UUID, PK)
└─ addon_group_id (FK → addon_groups)
└─ name (VARCHAR: "Chicken", "Fish")
└─ price (DECIMAL)
└─ is_available (BOOLEAN)
└─ sort_order

variant_groups -- for Shop service
└─ id (UUID, PK)
└─ item_id (FK → items)
└─ name (VARCHAR: "Color", "Size")
└─ sort_order

variant_options
└─ id (UUID, PK)
└─ variant_group_id (FK → variant_groups)
└─ name (VARCHAR: "Black", "XL")
└─ price_adjustment (DECIMAL, default 0)
└─ is_available (BOOLEAN)

```

Order Service

```

orders
└─ id (UUID, PK)
└─ order_number (VARCHAR, human-readable: "BSN-20240115-0001")
└─ customer_id (FK → users via API)
└─ store_id (FK → stores via API, NULL for parcel/errand)
└─ rider_id (FK → users via API, NULL until assigned)
└─ service_type (ENUM: food, groceries, shop, market, market_list, pharmacy, laundry, parcel, errand)
└─ order_type (ENUM: catalogue, request)

```

```

└─ order_type (ENUM: catalogue, express,
└─ status (VARCHAR – from state machine)
└─ payment_method (ENUM: mobile_money, card, cod, wallet)
└─ payment_status (ENUM: pending, authorized, captured, refunded, partially_refunded)
|
└─ item_total (DECIMAL)
└─ delivery_fee (DECIMAL)
└─ service_fee (DECIMAL)
└─ total_amount (DECIMAL)
|
└─ delivery_address_snapshot (JSON – copy of address at order time)
└─ delivery_latitude (DECIMAL)
└─ delivery_longitude (DECIMAL)
└─ delivery_landmark (TEXT)
└─ delivery_instructions (TEXT)
|
└─ pickup_address_snapshot (JSON – for parcels)
└─ pickup_latitude (DECIMAL, NULL for catalogue)
└─ pickup_longitude (DECIMAL, NULL for catalogue)
|
└─ prescription_image_url (VARCHAR, NULL – for pharmacy)
└─ errand_description (TEXT, NULL – for errands)
└─ errand_budget_items (JSON, NULL – for errands)
└─ special_instructions (TEXT)
|
└─ estimated_prep_time_minutes (INTEGER)
└─ estimated_delivery_time_minutes (INTEGER)
|
└─ placed_at (TIMESTAMP)
└─ accepted_at (TIMESTAMP)
└─ picked_up_at (TIMESTAMP)
└─ delivered_at (TIMESTAMP)
└─ cancelled_at (TIMESTAMP)
└─ cancellation_reason (TEXT)
└─ cancelled_by (ENUM: customer, vendor, rider, system, admin)
|
└─ is_cod (BOOLEAN)
└─ cod_collected (BOOLEAN, default false)
└─ cod_amount (DECIMAL)
|
└─ created_at

```

order_items

```

└─ id (UUID, PK)
└─ order_id (FK → orders)
└─ item_id (FK → items via API – for reference)
└─ item_name (VARCHAR – snapshot, in case vendor changes later)
└─ item_price (DECIMAL – snapshot)
└─ quantity (INTEGER)
└─ subtotal (DECIMAL)
└─ selected_addons (JSON – [{name, price}])
└─ selected_variants (JSON – [{group, option, price_adjustment}])
└─ special_instructions (TEXT)

```

order_status_history

```

└─ id (UUID, PK)
└─ order_id (FK → orders)
└─ from_status (VARCHAR)
└─ to_status (VARCHAR)
└─ changed_by (ENUM: customer, vendor, rider, system, admin)
└─ changed_by_user_id (UUID)
└─ note (TEXT, NULLABLE)
└─ created_at

```

```
wallets
├─ id (UUID, PK)
├─ user_id (UUID, UNIQUE)
├─ user_role (ENUM: customer, vendor, rider)
├─ balance (DECIMAL, default 0)
├─ cod_balance (DECIMAL, default 0 – riders only, what they owe)
├─ updated_at
└─ created_at

ledger_entries
├─ id (UUID, PK)
├─ transaction_id (UUID – groups related debit/credit pairs)
├─ account_type (ENUM: customer_wallet, vendor_wallet, rider_wallet, platform_revenue, platform_holding, platform_cash_holding, p
├─ account_id (UUID – wallet id or platform account id)
├─ entry_type (ENUM: debit, credit)
├─ amount (DECIMAL)
├─ balance_after (DECIMAL – running balance)
├─ reference_type (ENUM: order_payment, order_settlement, refund, withdrawal, cod_remittance, cod_obligation, commission, service
├─ reference_id (UUID – order_id, withdrawal_id, etc.)
├─ description (TEXT)
├─ idempotency_key (VARCHAR, UNIQUE)
└─ created_at

transactions
├─ id (UUID, PK)
├─ user_id (UUID)
├─ type (ENUM: payment, payout, refund, cod_remittance, settlement)
├─ amount (DECIMAL)
├─ payment_method (ENUM: mobile_money, card, wallet, bank)
├─ payment_provider (VARCHAR: "paystack")
├─ provider_reference (VARCHAR – Paystack transaction ref)
├─ status (ENUM: pending, success, failed)
├─ idempotency_key (VARCHAR, UNIQUE)
├─ metadata (JSON)
└─ created_at

withdrawals
├─ id (UUID, PK)
├─ user_id (UUID)
├─ wallet_id (FK → wallets)
├─ amount (DECIMAL)
├─ fee (DECIMAL, default 1.00)
├─ net_amount (DECIMAL – amount - fee)
├─ destination_type (ENUM: mobile_money, bank)
├─ destination_details (JSON – {provider, account_number, account_name})
├─ status (ENUM: pending, processing, completed, failed)
├─ provider_reference (VARCHAR)
└─ created_at
```

SECTION 18: EVENT FLOWS (Message Broker)

These are the key events that flow through RabbitMQ, triggering actions across services:

Order Lifecycle Events

```
order.placed
  → Notification Service: notify vendor (push + sound)
  → Payment Service: capture/authorize payment

order.vendor_accepted
  → Notification Service: notify customer
  → Dispatch Service: start rider assignment

order.vendor_rejected
  → Notification Service: notify customer
  → Payment Service: process full refund

order.rider_assigned
  → Notification Service: notify customer + vendor (with rider details)
  → Tracking Service: start tracking this rider for this order

order.picked_up
  → Notification Service: notify customer
  → Tracking Service: switch to delivery tracking mode

order.delivered
  → Payment Service: settle order (vendor earnings, rider earnings, platform revenue)
  → Notification Service: notify customer ("Your order has been delivered!")
  → User Service: increment vendor total_orders count

order.cancelled
  → Payment Service: process refund (based on cancellation rules)
  → Dispatch Service: release rider (if assigned)
  → Notification Service: notify affected parties
```

Rider Events

```
rider.location_updated
  → Tracking Service: store new location
  → Tracking Service: broadcast to customer (if active order)
  → Dispatch Service: update rider position for matching

rider.went_online
  → Dispatch Service: add to available rider pool

rider.went_offline
  → Dispatch Service: remove from available rider pool

rider.cod_remitted
  → Payment Service: clear COD obligation, settle funds
```

Payment Events

```
payment.success
  → Order Service: update payment_status to captured
  → Order Service: set order status to placed

payment.failed
  → Order Service: mark order as payment_failed
  → Notification Service: notify customer

refund.processed
  → Notification Service: notify customer
  → Order Service: update payment_status
```

SECTION 19: API ENDPOINT STRUCTURE (BFF Level)

These are the main endpoints your BFFs expose. NOT microservice-level endpoints – these are what the frontends actually call.

Customer BFF Endpoints

AUTH:

POST	/auth/otp/send	← send OTP to phone
POST	/auth/otp/verify	← verify OTP, get tokens
POST	/auth/google	← Google login
POST	/auth/apple	← Apple login
POST	/auth/refresh	← refresh access token
POST	/auth/phone/reverify	← sensitive action reverification

PROFILE:

GET	/me	← get customer profile
PATCH	/me	← update profile
GET	/me/addresses	← list saved addresses
POST	/me/addresses	← add address
PATCH	/me/addresses/:id	← update address
DELETE	/me/addresses/:id	← delete address

DISCOVERY:

GET	/home	← home screen data (aggregated)
GET	/services/:type/vendors	← list vendors for a service (food, groceries, etc.)
GET	/vendors/:id	← vendor detail + menu/catalogue
GET	/vendors/:id/items	← vendor items (paginated)
GET	/search?q=jollof&service=food	← search

CART & CHECKOUT:

GET	/cart	← get current cart
POST	/cart/items	← add item to cart
PATCH	/cart/items/:id	← update quantity/addons
DELETE	/cart/items/:id	← remove item
DELETE	/cart	← clear cart
GET	/checkout/summary	← price breakdown before payment
POST	/checkout	← place order + pay

ORDERS:

GET	/orders	← order history (paginated)
GET	/orders/:id	← order detail
GET	/orders/:id/track	← live tracking data (WebSocket upgrade)
POST	/orders/:id/cancel	← cancel order
POST	/orders/:id/topup/approve	← approve errand top-up
POST	/orders/:id/topup/reject	← reject errand top-up

REQUEST SERVICES:

POST	/parcels	← create parcel order
GET	/parcels/estimate	← get parcel price estimate
POST	/errands	← create errand order
GET	/errands/estimate	← get errand price estimate

WALLET:

GET	/wallet	← wallet balance
GET	/wallet/transactions	← transaction history
POST	/wallet/withdraw	← request withdrawal

CHAT:

GET	/orders/:id/messages	← chat history for an order
WS	/orders/:id/chat	← WebSocket for live chat

Vendor BFF Endpoints

AUTH: (same as customer)

STORE:

GET	/store	← get my store details
PATCH	/store	← update store details
PATCH	/store/status	← toggle open/closed
PATCH	/store/hours	← update operating hours

CATALOGUE:

GET	/categories	← list my categories
POST	/categories	← add category
PATCH	/categories/:id	← update category
DELETE	/categories/:id	← delete category
GET	/items	← list my items (paginated, filterable)
POST	/items	← add item
PATCH	/items/:id	← update item
DELETE	/items/:id	← delete item
PATCH	/items/:id/availability	← toggle in stock / out of stock
POST	/items/:id/addon-groups	← add addon group
PATCH	/addon-groups/:id	← update addon group
POST	/addon-groups/:id/addons	← add addon item
... (CRUD for addons and variants)		

ORDERS:

GET	/orders	← list orders (filterable by status)
GET	/orders/:id	← order detail
POST	/orders/:id/accept	← accept order
POST	/orders/:id/reject	← reject order (with reason)
POST	/orders/:id/status	← update status (preparing, ready, etc.)

EARNINGS:

GET	/earnings	← earnings summary
GET	/earnings/history	← detailed earnings history
GET	/wallet	← wallet balance
POST	/wallet/withdraw	← request withdrawal

DASHBOARD:

GET	/dashboard	← today's stats (orders, revenue, rating)
-----	------------	---

Rider BFF Endpoints

AUTH: (same as customer)

PROFILE:

GET	/me	← rider profile
PATCH	/me	← update profile
POST	/me/toggle-online	← go online / offline
POST	/me/location	← update GPS location

ASSIGNMENTS:

GET	/assignments/available	← current available assignment (if any broadcast pending)
POST	/assignments/:id/accept	← accept assignment
POST	/assignments/:id/decline	← decline assignment

ORDERS:

GET	/orders/active	← current active order
GET	/orders/:id	← order detail (with customer info, items, addresses)
POST	/orders/:id/status	← update order status
POST	/orders/:id/proof	← upload delivery proof / receipt photos
POST	/orders/:id/topup-request	← request top-up for errand

COD:

GET	/cod/balance	← current COD balance
POST	/cod/remit	← remit COD via mobile money

EARNINGS:

GET	/earnings	← earnings summary
GET	/earnings/history	← detailed history
GET	/wallet	← wallet balance
POST	/wallet/withdraw	← request withdrawal

CHAT:

GET	/orders/:id/messages	← chat history
WS	/orders/:id/chat	← WebSocket for live chat

SECTION 20: ORDER FLOW — END-TO-END WALKTHROUGH

To make absolutely sure everything connects, here is one complete order flow from start to finish.

Example: Customer Orders Food (Prepaid, Mobile Money)

CUSTOMER APP:

1. Kwame opens app → home screen loads (1 API call to Customer BFF /home)
2. Taps "Food" → sees list of restaurants near him (/services/food/vendors)
3. Taps "Auntie Adwoa's Kitchen" → sees menu (/vendors/:id)
4. Adds "Jollof Rice" with Chicken addon, qty 1 → POST /cart/items
5. Adds "Sobolo" drink, qty 2 → POST /cart/items
6. Taps "Checkout" → sees summary (/checkout/summary):
 - Jollof + Chicken: GHS 50
 - Sobolo × 2: GHS 20
 - Item total: GHS 70
 - Delivery fee: GHS 8
 - Service fee: GHS 3.50
 - Total: GHS 81.50
7. Confirms delivery address: "Osu, behind MTN mast, blue gate"
8. Selects payment: MTN Mobile Money
9. Taps "Place Order" → POST /checkout
 - Customer BFF → Order Service: create order (status: pending_payment)
 - Customer BFF → Payment Service: charge GHS 81.50 via Paystack
 - Paystack sends USSD prompt to Kwame's phone
 - Kwame approves on his phone
 - Paystack confirms → Payment Service creates ledger entries

- Payment Service → Order Service: payment confirmed
- Order Service: status → "placed"
- Order Service publishes event: order.placed

EVENT: order.placed

- Notification Service → Vendor push notification: "New order! 📢" (loud sound)
- Order visible in vendor app

VENDOR APP:

10. Auntie Adwoa sees new order on her dashboard
11. Taps order → sees items: Jollof + Chicken × 1, Sobolo × 2
12. Taps "Accept" → POST /orders/:id/accept
 - Order Service: status → "vendor_accepted"
 - Event: order.vendor_accepted
 - Notification Service → Customer: "Your order has been accepted!"
 - Dispatch Service: begins rider assignment

DISPATCH SERVICE (internal):

13. Finds 3 nearest online motorbike riders within 3km of vendor
14. Broadcasts assignment to all 3 simultaneously
15. Rider Mensah accepts first → POST /assignments/:id/accept
 - Order Service: status → "rider_assigned", rider_id = Mensah
 - Event: order.rider_assigned
 - Notification Service → Customer: "Mensah is your rider! 📢"
 - Notification Service → Vendor: "Rider Mensah will pick up the order"
 - Tracking Service: start tracking Mensah for this order

VENDOR APP:

16. Auntie Adwoa taps "Preparing" → status → "preparing"
 - Customer sees: "Your order is being prepared 📢"
17. Food is ready → taps "Ready for Pickup" → status → "ready_for_pickup"
 - Rider Mensah notified: "Order is ready! Head to Auntie Adwoa's"

RIDER APP:

18. Mensah navigates to vendor (sees vendor location on map)
19. Arrives → taps "Arrived at Vendor" → status → "rider_at_vendor"
 - Vendor sees: "Rider has arrived"
20. Collects food → taps "Picked Up" → status → "picked_up"
 - Customer notified: "Your food is on the way! 📢"
 - Live tracking activates – customer sees Mensah moving on map

CUSTOMER APP:

21. Kwame watches rider on map in real-time
22. ETA counting down: "8 minutes away"

RIDER APP:

23. Mensah navigates to Kwame's location (landmark: "blue gate behind MTN mast")
24. Needs clarification → opens chat: "I'm at the MTN mast, which direction?"
25. Kwame replies: "Turn left, second gate"
26. Mensah arrives → taps "Arrived" → status → "arrived"
 - Customer: "Your rider is here! 📢"
27. Hands over food → taps "Delivered" → status → "delivered"

EVENT: order.delivered

- Payment Service: settle order
 - Vendor wallet credited: GHS 59.50 (GHS 70 - 15% commission)
 - Rider wallet credited: GHS 8 (delivery fee)
 - Platform revenue: GHS 14 (GHS 10.50 commission + GHS 3.50 service fee)
 - Ledger entries created (debit/credit balanced)
- Notification Service → Customer: "Order delivered! Enjoy your meal 📢"
- Chat window: stays open for 30 more minutes, then closes

DONE. Total ledger movement: GHS 81.50 in = GHS 81.50 distributed. Zero discrepancy.

SECTION 21: WHAT IS NOT IN THIS PLAN (Intentionally Deferred)

These are important features that are **deliberately excluded from v1 core** to maintain focus. They will be added in subsequent phases after the core is stable.

Feature	Phase	Why Deferred
Ratings & reviews	Phase 2	Core flow works without it. Add after users are active.
Tips for riders	Phase 2	Nice-to-have, not blocking.
Promo codes & discounts	Phase 2	Marketing feature, not core flow.
Gift orders	Phase 2	Variation of normal order, easy to add.
Customer support chat	Phase 2	Use phone-based support at launch.
Vendor stories (WhatsApp-like)	Phase 3	Premium feature, needs media infrastructure.
Vendor premium subscription	Phase 2	All vendors are free at launch.
Referral program	Phase 2	Growth feature.
Order scheduling (future orders)	Phase 2	All orders are on-demand at launch.
Multi-vendor cart	Phase 3	Complex coordination, defer.
In-app VoIP calling	Phase 2	Masked phone numbers work for launch.
Detailed analytics for vendors	Phase 2	Basic stats (today's orders/revenue) only at launch.
Customer wallet top-up	Phase 2	Wallet receives refunds only at launch.
Surge pricing (automatic)	Phase 2	Manual peak pricing via admin at launch.
Rider incentives & bonuses	Phase 2	Manual bonus via admin at launch.
Dispute resolution system	Phase 2	Manual via admin at launch.
Email notifications	Phase 2	Push + SMS is sufficient for Ghana.

SECTION 22: DEVELOPMENT PRIORITY ORDER

If building all 8 services simultaneously, this is the order your team should build the **infrastructure and services**:

Sprint 1-2: Foundation

- ☑ API Gateway (auth, rate limiting, routing)
- ☑ Auth Service (phone OTP, Google, Apple, JWT)
- ☑ User Service (profiles, addresses)
- ☑ Customer BFF skeleton
- ☑ Vendor BFF skeleton
- ☑ Rider BFF skeleton
- ☑ Admin BFF skeleton
- ☑ Database setup (all service databases)
- ☑ Message broker setup (RabbitMQ)
- ☑ Redis setup

Sprint 3-4: Core Commerce

- 📦 Catalogue Service (stores, categories, items, addons, variants)
- 📦 Pricing Service (delivery fee, service fee, commission calculations)
- 📦 Order Service (cart, order creation, state machine)
- 📦 Payment Service (Paystack integration, ledger, wallets)
- 📦 Media Service (image upload, compression)

Sprint 5-6: Delivery Engine

- 📦 Dispatch Service (rider matching, broadcast, assignment)
- 📦 Tracking Service (GPS ingestion, live tracking, ETA)
- 📦 Notification Service (push, SMS)
- 📦 COD flow (obligation tracking, remittance)

Sprint 7-8: Communication & Search

- 📦 Chat Service (WebSocket, message storage)
- 📦 Search Service (full-text search, filters, sorting)
- 📦 Admin Service (dashboard, vendor/rider approval, order management)

Sprint 9-10: Integration & Testing

- 📦 End-to-end order flows (all 8 services)
- 📦 Payment flow testing (mobile money, card, COD)
- 📦 Real-time tracking testing
- 📦 Load testing
- 📦 Security audit
- 📦 Ghana-specific testing (network conditions, address edge cases)

This is the complete core plan for Besonc. Every gap has been filled. Every state machine defined. Every money flow accounted for. Every microservice scoped. Every endpoint listed. Your backend team knows what to build. Your frontend team knows what screens to create. Your architecture connects everything through BFFs and a message broker.

Build this, and you have a working multi-delivery platform for Ghana.